COAL Assignment 3 – Solution

EE-2003 Computer Organization and Assembly Language — Fall 2026

Name: Fawad Ali Roll No: _____ Section: BCS-3B

Q1 – Flags and Conditional Jumps

```
mov ax, 8000h
mov bx, 0001h
cmp ax, bx        ; computes ax - bx = 8000h - 0001h = 7FFFh (result
discarded)
```

a) Flags after CMP

Flag	Value	Reason
ZF	0	Result 7FFFh is not zero
SF	0	MSB of 7FFFh is 0
CF	0	Unsigned 8000h (32768) ⩾ 1, so no borrow
OF	1	Negative (−32768) − positive (1) gave a positive result, so signed overflow

b) Jumps

Jump	Condition	Taken?
JG	ZF = 0 and SF = OF	Not taken (SF ≠ OF)
JL	SF ≠ OF	Taken
JA	CF = 0 and ZF = 0	Taken
JB	CF = 1	Not taken

c) Why signed and unsigned jumps differ

The same bit pattern 8000h means 32768 when treated as unsigned, but −32768 when treated as signed. Unsigned jumps (JA, JB) use CF and ZF, so they see 32768 > 1. Signed jumps (JG, JL) use SF, OF and ZF, so they see −32768 < 1. CMP sets all the flags the same way; only the jump decides how to interpret them.

Q2 – Bubble Sort (Descending, Signed)

```
; Q2: bubble sort 8 signed words in descending order
[org 0x0100]
              jmp start

data:         dw  7, -2, 15, 4, -8, 10, 0, 3
swap:         db  0                   ; swap flag for current pass
swapCount:    dw  0                   ; total number of swaps

start:        mov  bx, 0              ; start from first element
              mov  byte [swap], 0     ; reset swap flag

loop1:        mov  ax, [data+bx]      ; load current element
              cmp  ax, [data+bx+2]    ; compare with next element
              jge  noswap             ; signed: already in descending order

              mov  dx, [data+bx+2]    ; swap the two elements
              mov  [data+bx+2], ax
              mov  [data+bx], dx
              mov  byte [swap], 1     ; a swap was done in this pass
              inc  word [swapCount]   ; count the swap

noswap:       add  bx, 2              ; next index
              cmp  bx, 14             ; last pair is at index 14
              jne  loop1

              cmp  byte [swap], 1     ; any swap in this pass?
              je   start              ; yes, do another pass

              mov  ax, 0x4c00         ; terminate program
              int  0x21
```

- JGE is a signed jump, so negative numbers are compared correctly. Using JGE (not JG) also avoids swapping equal elements.
- BX stops at 14 because 8 words occupy indexes 0–14, and the last pair compared is at index 12 and 14.
- Sorted array: 15, 10, 7, 4, 3, 0, −2, −8
- swapCount = 12 (000Ch)

Q3 – Shift and Rotate

AL = 10110101b, CF = 0 (each operation done independently)

Operation	AL after	CF
a) SHR AL, 1	01011010b	1
b) SHL AL, 1	01101010b	1
c) SAR AL, 1	11011010b	1
d) ROR AL, 1	11011010b	1
e) RCL AL, 1	01101010b	1

- SHR: bits move right, 0 enters MSB, LSB (1) goes to CF.
- SHL: bits move left, 0 enters LSB, MSB (1) goes to CF.
- SAR: bits move right, MSB (sign bit 1) is copied, LSB goes to CF.
- ROR: LSB (1) goes to both MSB and CF.
- RCL: MSB (1) goes to CF, old CF (0) enters LSB.

Difference:

- Logical shift (SHL/SHR): fills the vacated bit with 0; used for unsigned numbers. SHR divides an unsigned number by 2.
- Arithmetic shift (SAR): keeps the sign bit when shifting right, so a signed number is divided by 2 correctly.
- Rotation (ROL/ROR/RCL/RCR): no bit is lost; the bit that leaves one end comes back at the other end (RCL/RCR pass it through CF).

Q4 – Shift-and-Add Multiplication (25 × 13)

```
; Q4: 8bit x 8bit unsigned multiplication using shift and add
[org 0x0100]
              jmp start

multiplicand: dw  25                  ; 8bit multiplicand (16bit space)
multiplier:   db  13                  ; 8bit multiplier
result:       dw  0                   ; 16bit result

start:        mov  cl, 8              ; 8 bits to check
              mov  bx, [multiplicand] ; load multiplicand in bx
              mov  dl, [multiplier]   ; load multiplier in dl

checkbit:     shr  dl, 1              ; move right most bit in carry
              jnc  skip               ; skip addition if bit is zero
              add  [result], bx       ; accumulate result
```

```
skip:         shl  bx, 1              ; shift multiplicand left
              dec  cl                 ; decrement bit count
              jnz  checkbit           ; repeat if bits left

              mov  ax, 0x4c00         ; terminate program
              int  0x21
```

The multiplicand is kept in 16-bit space because after 8 left shifts it can grow beyond 8 bits.

Multiplier 13 = 00001101b, so bits 0, 2 and 3 are 1:

Bit	CF	Multiplicand (BX)	Result
0	1	25	25
1	0	50	25
2	1	100	125
3	1	200	325
4–7	0	–	325

result = 325 = 0145h

Q5 – Extended (32-bit) Arithmetic

num1 = 0000FFFFh → upper word 0000h, lower word FFFFh

num2 = 00000002h → upper word 0000h, lower word 0002h

a) Addition

```
Lower:  FFFFh + 0002h       = 1 0001h  -> lower = 0001h, CF = 1   (ADD)
Upper:  0000h + 0000h + CF  = 0001h                              (ADC)
Result = 0001 0001h
```

```
mov ax, [num1]
add ax, [num2]        ; lower words, CF = 1
mov dx, [num1+2]
adc dx, [num2+2]      ; upper words + carry
```

b) Subtraction num1 – num2

```
Lower:  FFFFh - 0002h       = FFFDh   -> CF = 0 (no borrow)   (SUB)
Upper:  0000h - 0000h - CF  = 0000h                           (SBB)
Result = 0000 FFFDh
```

```
mov ax, [num1]
sub ax, [num2]        ; lower words, CF = 0
mov dx, [num1+2]
sbb dx, [num2+2]      ; upper words - borrow
```

c) Purpose of ADC and SBB

A 16-bit register cannot hold a 32-bit number, so the operation is done one word at a time, starting from the lower word. ADD/SUB on the lower words sets CF when a carry or borrow comes out of bit 15.

- ADC (Add with Carry): dest = dest + src + CF. It adds the carry from the lower word into the upper word.
- SBB (Subtract with Borrow): dest = dest − src − CF. It subtracts the borrow from the lower word out of the upper word.

The same idea extends to any size (48, 64 bits…) by using ADC/SBB for every word after the first.

Q6 – 32-bit × 32-bit Multiplication (64-bit Product)

The shift-and-add algorithm from the handouts is extended to 32 bits. The multiplicand is kept in 64-bit space so it can be shifted left 32 times, and the result is a 64-bit memory location. Only the four allowed GPRs are used (EAX, ECX, EDX; EBX is not needed). Sample sensor readings are used.

```
; Q6: 32bit x 32bit unsigned multiplication, 64bit product
[org 0x0100]
             jmp start

multiplicand: dd  0x12345678, 0       ; 32bit multiplicand (64bit space)
multiplier:   dd  0x00ABCDEF          ; 32bit multiplier
result:       dd  0, 0                ; 64bit result

start:        mov  ecx, 32            ; 32 bits to check
              mov  edx, [multiplier]  ; load multiplier in edx

checkbit:     shr  edx, 1             ; move right most bit in carry
              jnc  skip               ; skip addition if bit is zero

              mov  eax, [multiplicand]
```

```
                add  [result], eax      ; add lower dword
                mov  eax, [multiplicand+4]
                adc  [result+4], eax    ; add upper dword with carry

skip:           shl  dword [multiplicand], 1
                rcl  dword [multiplicand+4], 1 ; shift 64bit multiplicand left
                dec  ecx                ; decrement bit count
                jnz  checkbit           ; repeat if bits left

                mov  ax, 0x4c00         ; terminate program
                int  0x21
```

- SHR EDX, 1 puts the next multiplier bit in CF.
- If CF = 1, the 64-bit multiplicand is added to the 64-bit result using ADD (lower dword) and ADC (upper dword).
- SHL/RCL shift the 64-bit multiplicand left; the bit leaving the lower dword enters the upper dword through CF.
- For the sample values, result = 12345678h × 00ABCDEFh = 000C379A AA42D208h.

(Note: the single instruction `mul ebx` would also give the product in EDX:EAX, but the shift-and-add method is shown as it follows the course algorithm.)